

Model Cell Explanations

Our model predicts small gap errors between actual and theoretical output. That gap gets added to the physics baseline and scaled by capacity to produce a real kw predicted generation.

Part 1: Model -

- Cell 1: Imports the dataset, creates the cyclical features where we extract the minutes, days, and hours. We use the number 365.25 instead of 365 to take into account leap years. We use cyclical features because it allows the model to take in numbers and map them as coordinates on the unit circle to learn about time. Since the dataset was hard coded with the number 1000 for plant capacity, we reverse engineered to obtain the actual capacity. We want a ratio that is from 0-1 so for example lets say the actual capacity of the plant was 1000, that means our actual ac power generated at most can be 1000. The reason for this is because when we do the ratio of actual ac power over capacity we would get $1000/1000$ which is 1 meaning that the plant is generating power at 100%. We retrieve the theoretical ac per kwp above 0.5 since this is near peak sun hours and makes sure that our ratio is stable. In order to get the implied capacity of the plant we have to use the formula of actual ac power in kw over theoretical ac per kwp. Taking the median of this allows us to ignore extreme outliers. We then get our actual ac per kwp by using the formula of actual ac power over the implied capacity. After this we find our target variable being the gap error by subtracting the actual ac per kwp by the theoretical ac per kwp. We make sure our values are right by generating some output and then creating our input features being irradiation, ambient temperature, wind speed, cloud cover, time of day, and day of year. The output of this cell shows us the huge mismatch between the hard coded 1000 and the implied capacity of the plant revealing the capacity was actually

20,230 kwp. We also see the stats of our target variable which shows a mean of 0.0331 meaning the physics formulas underestimate real-world generation by 3.3% roughly. The standard deviation shows that for most daylight hours, the physics model's error stays within a window of 0.1212 kw/kwp from the average. The min of -0.6784 is the worst overestimation where the physics predicted way too much power where the real output dropped 0.6784 kw/kwp below what the formula expected. The max of 0.4362 is the worst underestimation where the panels performed much better than what the physics predicted.

- Cell 2: We trained our model using XGBoost regression as it allows the model to fix its mistakes step by step making it significantly better at predicting continuous numbers such as gaps. We used an 80/20 split meaning 80% of our data was used to train the data while the other 20% was used to test. We ended up with an accuracy score of 81%, a root mean square value of 0.0492, and a mean absolute error of 0.0206.
- Cell 3: We then evaluated our model by using a 5-Fold Cross test as it allows us to see if our accuracy score holds. This type of test is when we take portions of our data in our dataset where we then separate it into folds where they're used to train and test the model. We do this 5 times and get an accuracy score for each fold. Our mean accuracy score was found to be 75% where the highest was fold 1 at 80% and fold 5 being the lowest at 62%.
- Cell 4: In this cell we have two functions. The first function has it so that our model predicts the gap error where we then add this gap to the theoretical ac per kwp values. This combined value represents our actual ac per kwp as we know from earlier that $\text{gap} = \text{actual} - \text{theoretical}$, so now we're doing $\text{actual} = \text{gap} + \text{theoretical}$. We then force night

rows to be 0 and negative values to also be 0. The second function scales this value up by multiplying by the plant's capacity converting the value from kwp to kw.

- Cell 5: In this cell we are mirroring the physicsCalc.py where we have variables that are defaulted to the real world standards. For example, the default temperature coefficient is -0.004 because in the real world once a panel's module temperature exceeds that of 25 degrees Celsius the panel loses efficiency at a rate of 0.4% for every degree above 25 degrees Celsius. The first function calculates the module temperature of the panel which is crucial as we know that panels lose efficiency the hotter they get. The primary equation from this function is the ambient temperature + any extra heat from the sun being delta t. The function after this calculates how much dc power a panel generates per kwp. The third function calculates the ac power per kwp obtained by multiplying the dc power per kwp by the inverter efficiency. The last function in this cell is important because it converts the time values into a single row DataFrame which makes it easy for our model to read. The second part of this cell is some tests that if irradiation is 0 then the output generation should also be 0 and the other test makes sure that the warmer panels are losing more efficiency than colder panels.
- Cell 6: This cell evaluates the model by running a backtest. It does this by testing the model with data it hasn't seen before. We extract the physics baseline values and the irradiation which will be used to enforce night time zero rules. We also extract the actual ac power and the timestamps so we can plot the predictions chronologically. We feed the features into the XGBoost model to obtain the gap errors and add them to the physics baseline, enforce the night guardrails, and scale the output by the plant capacity to get the final predicted kw. We then calculate the error for every single hour and the metrics. The

left plot checks if errors get worse over time or during specific months. The right plot checks if errors get worse during low power and high power. The combined physics and model reconstructs real kw with an accuracy score of 97%. This is not directly comparable to the 81%/75% accuracy scores from before since those measure the model's skill on the small per kwp gap, while this is inflated by the physics baseline.

- Cell 7: Summary page of all of our metrics so far
- Cell 8: This cell retrains the model once more on all data this time not 80% since we want the model to be exposed to as much information as possible. We then save the final production model.

Part 2: Application -

- Cell 9: This cell contains a function that builds and collects the user input where location, plant capacity, and time are required while the other features are optional. If the optional fields are left blank then they will be replaced with default standard values. We also have a user tier toggle for simple or advanced output depending on who the user is. We have an example advanced input for Chennai, India with a time range of 14 days. The time range of days matters because if it's less than or equal to 10 days we provide a forecast and if it's greater than 10 days then we fetch historical years worth of data. We also keep track of how many optional inputs were left blank.
- Cell 10: This cell converts the user's input into geographic coordinates. We then use these coordinates to resolve the two optional fields of tilt and azimuth depending on where the system is located. A rule of thumb in solar design is that a fixed panel captures roughly the most annual sunlight when angles close to the plants latitude. This is why we default the tilt to the latitude. In the northern hemisphere, the sun's arc spends most of

the day towards the south, so a south facing panel collects the most sunlight over a full day. We used the weather api's convention where azimuth 0 = south. In the southern hemisphere the logic flips. This is why azimuth defaults to 0 degrees in the northern hemisphere and 180 degrees in the southern hemisphere.

- Cell 11: This cell branches on the user's chosen time and day range where it's less than or equal to 10 days, we call the forecast API for those dates. If it's greater than 10 days we call the historical archive API, pulling up to 10 past years of real weather for the matching calendar window. In this cell we also handle leap years. For example, if the target date range happens to include February 29 and a historical year being requested isn't a leap year itself we fall back to February 28 in order to keep the loop running.
- Cell 12: We start off with a function that builds the time feature that converts the times and days as coordinates on the unit circle. Then we have a function that runs the entire prediction pipeline where we scale by multiplying by the user's capacity.
- Cell 13: This cell retrieves the weather data and runs the prediction pipeline. If requested, we compare the results against past years historical weather data. The variable `primary_df` is our primary forecast table while `by_year_results` is a dictionary of past years table.
- Cell 14: This cell computes two things: first being computing a confidence label based on how many optional input fields were left blank and second being an out of range flag if the user's input is outside our training data's period or location. We also flag out of range if the target latitude is more than 15 degrees away from the training plant (8.85 degrees North). We use 15 degrees as the bounds since once a requested location is more than about 15 degrees of latitude away from the training plants, the weather patterns start to

look different. We obtained the 8.85 degrees from our plant by using sunrise/sunset times which depend on our latitude and day of the year.

- Cell 15: This cell contains our monthly bar chart. The monthly bar chart is a bar for each month of the year showing expected output, letting the user see the seasonal shape at a glance. This cell aggregates hourly predictions across all retrieved historical years converting kilowatts into average monthly kilowatt hours.
- Cell 16: This cell contains our day in the life curve. This is a single sample day's generation curve, rising in the morning, peaking midday, and falling in the evening, giving an intuitive visual of how a typical day behaves. The gray dashed lines show us the physics baseline while the orange solid line shows us what our model predicted.
- Cell 17: This cell contains our risk rouge gauge. This is a simple visual showing the gap between the worst realistic case and the typical case, labeled in plain words rather than statistical terms for the simple tier. This bar gives us our P90 and P50 values. The P90 is the conservative worst case scenario value where we have a 90% of exceeding this value. The P50 value gives us the annual average where we have a 50% of exceeding this value. This is only for when the user fetches for multiple historical years.
- Cell 18: This cell contains our quantile table. This is a table for the advanced tier showing the full P10 through P90 breakdown, by month and by year.
- Cell 19: This cell contains our multiple day types curves. This cell gives us separate generation curves for a clear day, a cloudy day, and a rainy or monsoon like day, so the advanced user can see the range of daily behavior rather than just one example.
- Cell 20: This cell contains our model confidence panel. This is a small section showing the model's own backtested error range, kept visually separate from the weather risk

range so the two types of uncertainty are not confused. It shows how far off the model's predictions were from the actual measured power. The huge spike right at zero means the majority of predictions were very close to the actual measurements.

- Cell 21: This contains our degradation projection chart. This is a chart comparing year one output against a later year, such as year ten, adjusted for panel aging, shown only if the user provided a degradation rate.
- Cell 22: This cell contains our transparency table. This is a full expandable table of every underlying value used to build the outputs, timestamp, weather inputs, physics number, model correction, and final number, available to any user who wants to dig deeper. Negative model corrections means that the final prediction ended up lower than the physics baseline. Positive model corrections means that the final prediction was higher than the physics baseline.
- Cell 23: This cell contains our suggestions panel. These are short actionable notes tied to the user's own numbers, for example suggesting appliance timing for a simple tier user or noting financial planning considerations for an advanced tier user.
- Cell 24: This cell contains the risk and caveats panel. This is a dedicated space listing anything relevant to trustworthiness, including the out of range warning, the list of which fields were defaulted, and a plain explanation of what P50 and P90 mean.
- Cell 25: This cell contains the input summary tab which is a tab or panel showing back exactly what the user entered and what was defaulted, so they can review and correct it without having to redo the whole input process.

- Cell 26: This cell contains the comparison tab which is an optional tab letting the user compare two different input scenarios side by side, for example comparing their system at two different capacities or two different locations.
- Cell 27: This cell allows us to see which input factor contributes the most to our output. The left chart shows what the model thinks is important while learning while the right chart shows what's actually important when tested. We shuffle one feature column at a time on unseen test data and measure the impact based on the drop in the accuracy score. The sharper the drop in accuracy, the more critical the feature is.